WEJ LAB

Neural Networks and Deep Learning

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-11-11

RUTGERS HEALTH
Center for Data Science

WEJ LAB

"Biology / Biomedicine is a Data Science."

— W. Evan Johnson

Addendum: don't avoid the math!

RUTGERS HEALTH
Center for Data Science

WEJ LAB

"With great power comes great responsibility."

— Uncle Ben, *Spider-Man*

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Describe an artificial neuron: weights, bias, activation function
- Explain how a network learns: loss, backpropagation, gradient descent
- Fit and tune a multilayer perceptron in R (`neuralnet`, `nnet` via `caret`)
- Explain what convolutional and recurrent networks add, and why
- Describe the key idea of transformers: attention

A quick map of “AI”

Where do neural networks fit in the broader AI landscape?

- **Narrow AI** (all AI today): does one specific task well — Face ID, spam filters, recommendations, Google Maps, ChatGPT
- **Generative AI**: creates new content from learned patterns — text, images, music, video, voice
- **Agentic AI** (emerging): takes actions, makes plans, and uses tools to accomplish goals — schedules meetings, sends email, runs tasks automatically

A quick map of "AI"

- **Artificial General Intelligence (AGI)**: human-level reasoning across many tasks — **theoretical; does not exist** (movies exaggerate it)
- **Artificial Super Intelligence (ASI)**: hypothetical intelligence that would *surpass* humans across virtually all domains — **also does not exist**

Everything we build today — including the neural networks in this lecture — is **Narrow AI**.

Real (human) intelligence

The ability to think, choose, create, understand meaning, and act with moral responsibility:

- **Agency** — we choose
- **Moral judgment** — right vs. wrong
- **Creativity** — truly new ideas
- **Understanding meaning**
- **Empathy and relationships**
- **Spiritual capacity**

WEJ LAB
RUTGERS HEALTH
Center for Data Science

WEJ LAB

"AI will not replace us, it will amplify us."

— Pedro Domingos

RUTGERS HEALTH
Center for Data Science

WEJ LAB

From neurons to networks

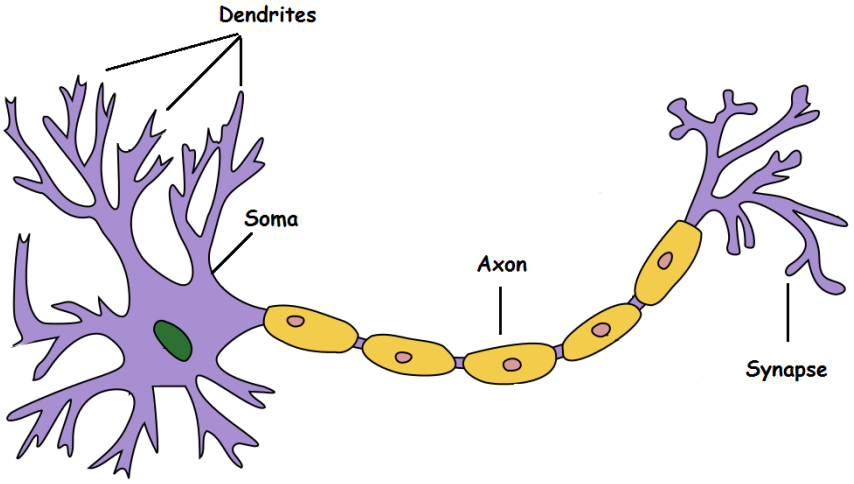
RUTGERS HEALTH
Center for Data Science

Biological neurons

The brain has billions of neurons, each a simple processor, wired into a **biological neural network** that passes electrical signals from neuron to neuron:

- **Dendrites** — receive input signals
- **Soma** (cell body) — sums the inputs
- **Axon** $\rightarrow$ **synapses** — pass the output on to other neurons

Biological neurons



WEJ LAB
TGERS HEALTH
nter for Data Science

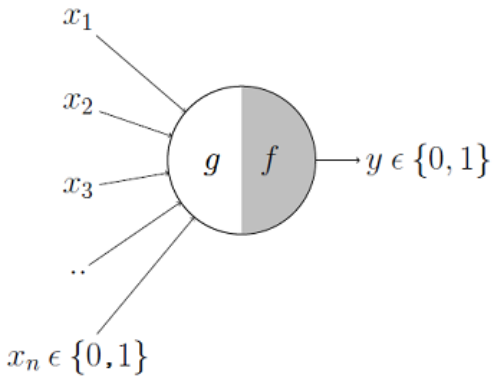
McCulloch & Pitts (1943)

The first mathematical model of a neuron:

- Binary inputs, fixed weights, and a **threshold**: fire (1) or not (0)
- **No layers and no training rule**
- Networks of such units can compute logical functions: the brain as a computing device

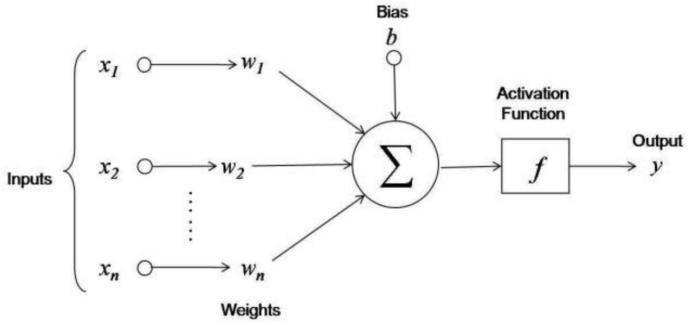
WEJ LAB
RUTGERS HEALTH
Center for Data Science

McCulloch & Pitts (1943)



The perceptron (Rosenblatt, 1957–58)

The first **trainable** neural network: one layer of adjustable weights and a threshold, learned from labeled examples.



A short history

- **1943:** McCulloch & Pitts neuron
- **1958:** Rosenblatt's perceptron
- **1969:** Minsky & Papert, *Perceptrons*: a single-layer perceptron can't learn simple functions like XOR $\rightarrow$ interest collapses
- **1986:** Rumelhart, Hinton & Williams popularize **backpropagation** for multilayer networks
- **2012:** AlexNet (deep CNN + GPUs) wins ImageNet $\rightarrow$ the **deep learning** era

An artificial neuron

$$\text{output} = g\Big(\sum_{j=1}^{p} w_j x_j + b\Big)$$

- **Inputs** $x_1, \ldots, x_p$: the predictors
- **Weights** w_j and **bias** b: the parameters we learn
- **Activation function** g: maps the weighted sum to the output

With g = logistic, a single neuron **is logistic regression**.

Activation functions

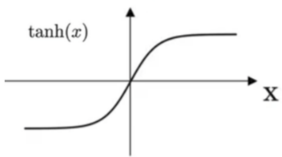
The activation function gives the network its **non-linearity**:

- **Identity**: output = input (linear; regression outputs)
- **Binary step**: on/off above a threshold (the original perceptron)
- **Sigmoid / tanh**: smooth S-curves; sigmoid gives probabilities
- **ReLU** $\max(0, x)$: the modern default for hidden layers
- **Softmax**: turns K outputs into class probabilities

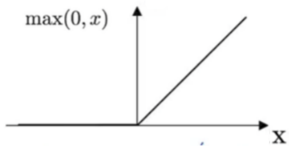
WEJ LAB
RUTGERS HEALTH
Center for Data Science

Activation functions

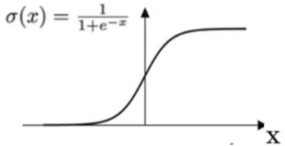
Hyper Tangent Function



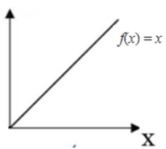
ReLU Function



Sigmoid Function

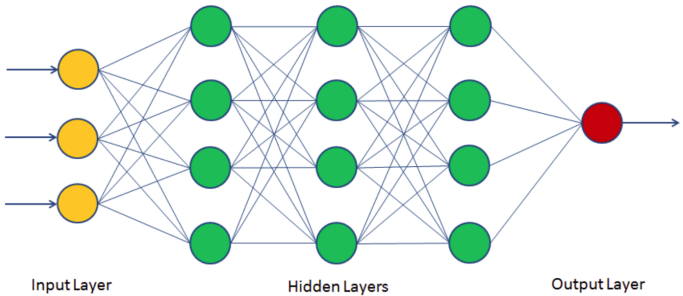


Identity Function



Multilayer networks

A **multilayer perceptron** (MLP) stacks layers of neurons: input $\rightarrow$ hidden layer(s) $\rightarrow$ output. "**Deep**" = many hidden layers.



WEJ LAB

How networks learn

RUTGERS HEALTH
Center for Data Science

How does a neural network learn?

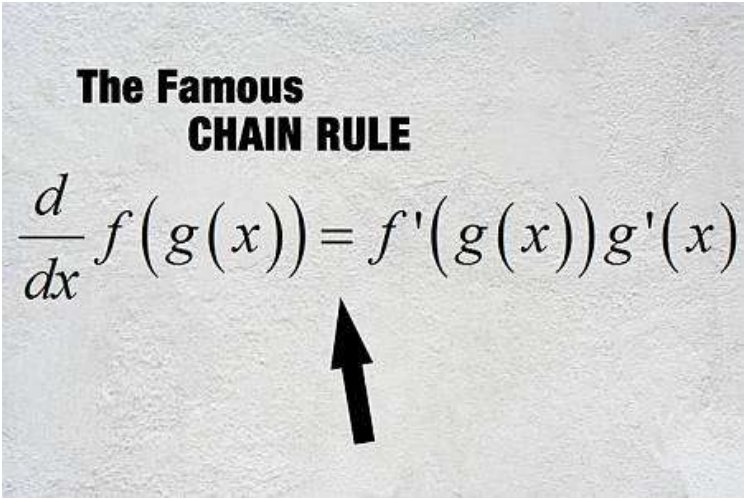
Training means finding the **weights** that make predictions match the data:

1. A **loss** $\ell(y, \hat{y})$: how wrong is the prediction? (squared error, cross-entropy)
2. **Backpropagation**: how does each weight affect the loss?
3. **Gradient descent**: nudge each weight to reduce the loss

"AI is not magic; it's math."

— Peter Norvig

The chain rule



WEJ LAB
RUTGERS HEALTH
Center for Data Science

The chain rule

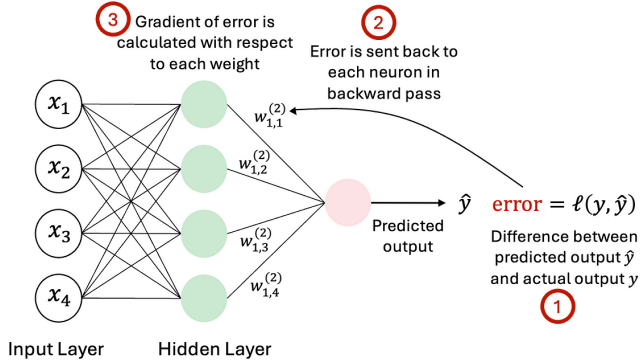
A neural network is **functions composed inside functions**. The derivative of a composition multiplies the derivatives of each layer:

$$f' \circ g \circ h \circ k(x) \times g' \circ h \circ k(x) \times h' \circ k(x) \times k'(x)$$



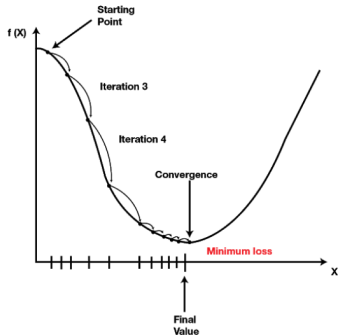
Backpropagation

The chain rule sends the error **backward** through the network, giving the gradient of the loss for **every weight**:



Gradient descent

Update each weight against its gradient: $w \leftarrow w - \eta \, \partial \ell / \partial w$ (learning rate η). In practice, **stochastic** gradient descent uses small **mini-batches**; one pass through the data is an **epoch**.



Training in practice

Networks have many weights, so they **overfit** easily:

- **Weight decay** (L2 penalty): regularization again (`decay` in `nnet`)
- **Early stopping**: stop when validation loss stops improving
- **Dropout**: randomly switch off neurons during training
- **Scale the inputs**; start from **random weights** (results vary by seed)

WEJ LAB

Neural networks in R

RUTGERS HEALTH
Center for Data Science

A toy example: job placement

Technical knowledge (TKS) and communication skill (CSS) scores (rescaled to 0–1), and placement:

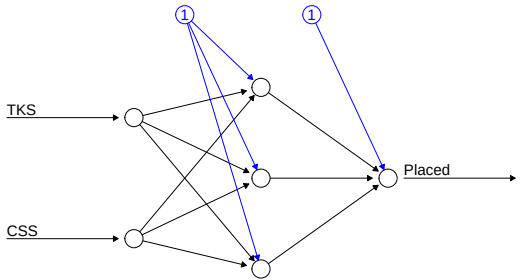
```
df <- data.frame(TKS    = c(20, 10, 30, 20, 80, 30) / 100,
                 CSS    = c(90, 20, 40, 50, 50, 80) / 100,
                 Placed = c(1, 0, 0, 0, 1, 1))
t(df)
```

```
##         [,1] [,2] [,3] [,4] [,5] [,6]
## TKS      0.2  0.1  0.3  0.2  0.8  0.3
## CSS      0.9  0.2  0.4  0.5  0.5  0.8
## Placed   1.0  0.0  0.0  0.0  1.0  1.0
```

A toy example: job placement

A multilayer perceptron with one hidden layer of 3 neurons:

```
set.seed(0)
nn <- neuralnet(Placed ~ TKS + CSS, data = df, hidden = 3,
                act.fct = "logistic", linear.output = FALSE)
```



A toy example: job placement

Predict for three new students; threshold at 0.5:

```
test <- data.frame(TKS = c(30, 40, 85) / 100, CSS = c(85, 50, 40) / 100)
prob <- predict(nn, test)
cbind(test, prob = round(prob, 3), pred = ifelse(prob > 0.5, 1, 0))
```

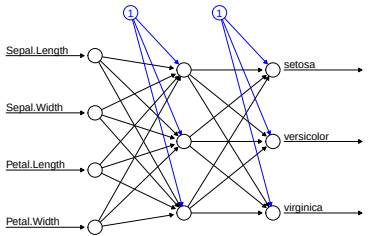
```
##    TKS  CSS  prob pred
## 1 0.30 0.85 0.942    1
## 2 0.40 0.50 0.442    0
## 3 0.85 0.40 0.903    1
```

With 6 training points, this is an illustration, not a model!

Iris: three classes

`neuralnet` fits one output neuron per class (inputs scaled first; unscaled, it fails to converge):

```
iris_s <- mutate(iris, across(where(is.numeric), ~ as.numeric(scale(.x))))
set.seed(0)
nn_iris <- neuralnet(Species ~ ., data = iris_s, hidden = 3,
                     linear.output = FALSE)
plot(nn_iris, rep = "best", information = FALSE, show.weights = FALSE)
```



Tuning with caret: `nnet`

`method = "nnet"`: one hidden layer; tune its **size** and the weight **decay**.
Train/test split, as always:

```
set.seed(0)
idx <- createDataPartition(iris$Species, p = 0.7, list = FALSE)
nn_caret <- train(Species ~ ., data = iris[idx, ], method = "nnet",
                  preProcess = c("center", "scale"), trace = FALSE)
nn_caret$bestTune
```

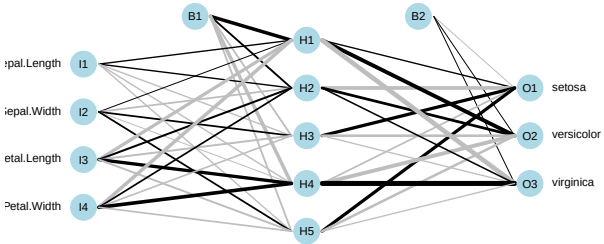
```
##   size decay
## 9    5   0.1
```

```
confusionMatrix(predict(nn_caret, iris[-idx, ]),
                iris$Species[-idx])$overall["Accuracy"]
```

```
##  Accuracy
## 0.9555556
```

Tuning with caret: nnet

```
NeuralNetTools::plotnet(nn_caret$finalModel)
```



Biomarker development revisited: TB

The TB NanoString data from the SVM lecture, same split and CV:

```
tb <- readRDS("TBnanostring.rds")
tb$TB_Status <- factor(tb$TB_Status, levels = c("LTBI", "TB"))
set.seed(1)
idx <- createDataPartition(tb$TB_Status, p = 0.7, list = FALSE)
tb_train <- tb[idx, ]; tb_test <- tb[-idx, ]
ctrl <- trainControl(method = "cv", number = 10, classProbs = TRUE,
                     summaryFunction = twoClassSummary)
set.seed(1)
nn_tb <- train(TB_Status ~ ., data = tb_train, method = "nnet",
               preProcess = c("center", "scale"), metric = "ROC", trControl = ctrl,
               tuneGrid = expand.grid(size = c(1, 3, 5), decay = c(0.01, 0.1, 1, 2)),
               trace = FALSE, maxit = 500)
nn_tb$bestTune
```

```
##   size decay
## 8    3     2
```

Biomarker development revisited: TB

```
p_tb <- predict(nn_tb, tb_test, type = "prob")[, "TB"]
auc(roc(tb_test$TB_Status, p_tb, levels = c("LTBI", "TB"),
        direction = "<", quiet = TRUE))
```

```
## Area under the curve: 0.9319
```

```
confusionMatrix(predict(nn_tb, tb_test), tb_test$TB_Status,
                positive = "TB")$byClass[c("Sensitivity", "Specificity")]
```

```
## Sensitivity Specificity
##   0.8695652   0.9333333
```

Test AUC $\approx$ the SVMs (0.92–0.93), with **strong weight decay** chosen. With 125 samples and 107 genes, a neural net has no edge.

When does deep learning shine?

- **Large** data sets (thousands to millions of examples)
- **Raw, structured inputs**: images, sequences, text, audio
- The network **learns the features** that we would otherwise hand-engineer
- For small **tabular** data (most clinical and omics studies), regularized regression, forests, and boosting are usually as good and easier to tune

WEJ LAB

Network architectures

RUTGERS HEALTH
Center for Data Science

Feedforward vs. recurrent

- **Feedforward**: signals travel one way, layer to layer, input to output (the MLP)
- **Recurrent**: contains a loop: the hidden state from the previous step is fed back in, giving the network **memory**
- **Convolutional**: shares weights across positions in an image or sequence

Others: LSTM/GRU, sequence-to-sequence, radial basis function networks, autoencoders, transformers.

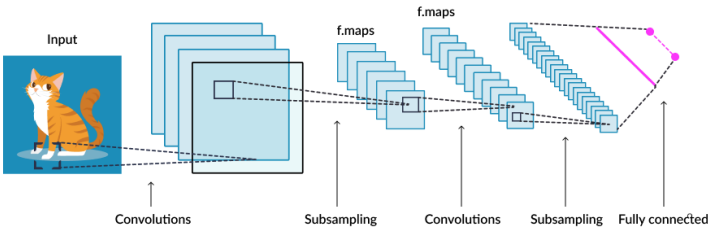
Convolutional neural networks

A **CNN** is specialized for **grid-like data** such as images:

- Small **filters (kernels)** with **shared weights** slide across the input $\rightarrow$ **feature maps**
- A pattern is detected **wherever it appears** (translation equivariance; pooling adds approximate invariance)
- Far fewer weights than a fully connected MLP: a built-in **regularization**

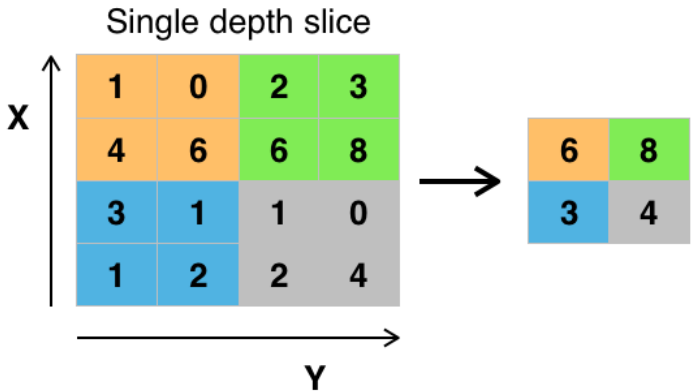
Convolutional neural networks

- **Filters are learned** from data, not hand-engineered
- Early layers find edges; later layers find shapes and objects
- Uses: medical imaging, pathology slides, video, genomic sequences



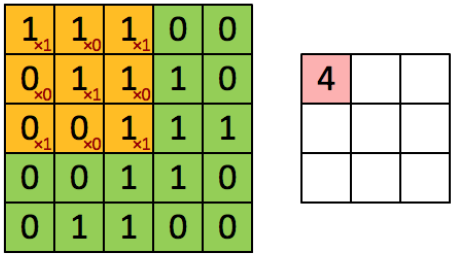
Convolutional neural networks

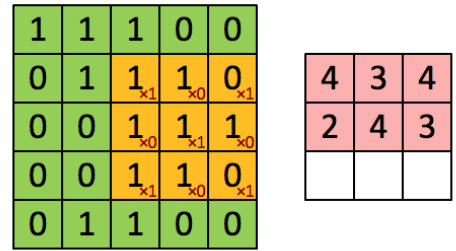
Pooling (here, max pooling) shrinks each feature map:



Convolutional neural networks

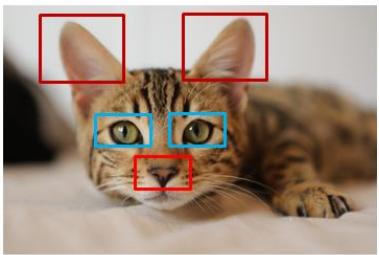
A sliding (overlapping) filter: **kernel**, **stride**, and **padding**:

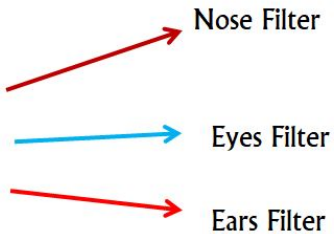




Convolutional neural networks

Now applying this to cats:





Convolutional neural networks

More realistically:



Cat



Cat



Representation

Convolutional neural networks

And as a classifier:





Recurrent neural networks

- For **sequential data**: text, time series, speech, sequences
- Standard networks assume **independent inputs**, but sequences have **dependencies**: in *"The cat sat on the ____"*, you need the previous words
- A **hidden state** carries memory from one step to the next

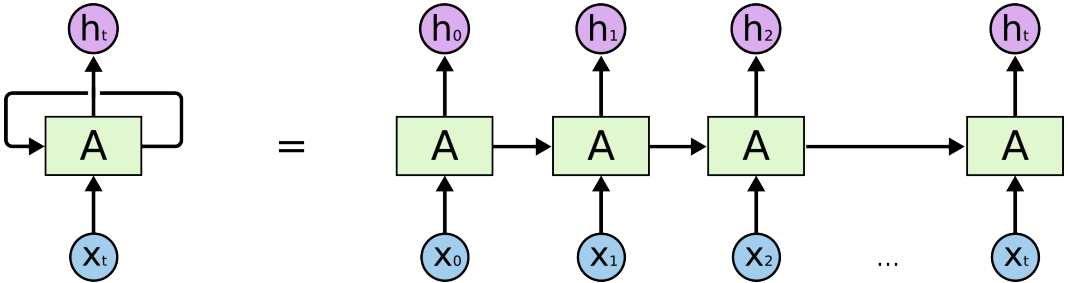
RNN architecture

The same weights are applied at every time step t:

- Input: x_t
- Hidden state: $h_t = f(Wx_t + Uh_{t-1} + b)$
- Output: $y_t = g(Vh_t + c)$

WEJ LAB
RUTGERS HEALTH
Center for Data Science

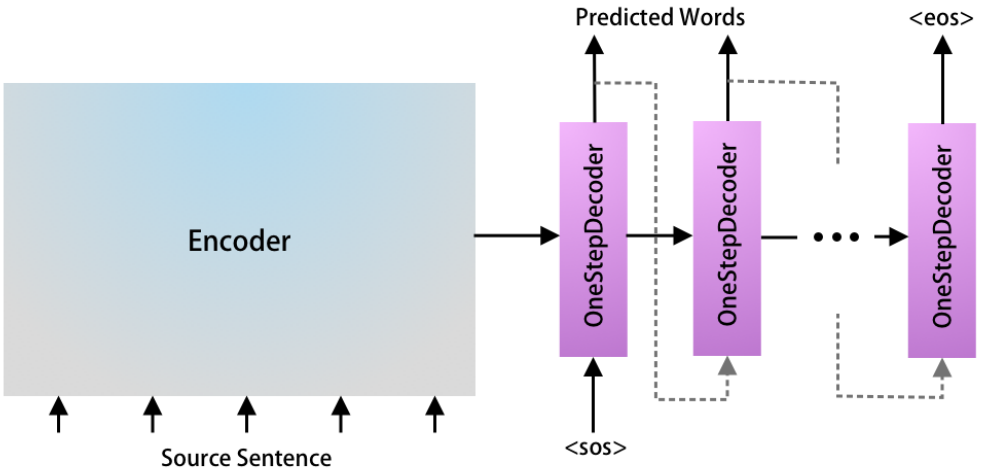
Diagram of an RNN (unrolled)



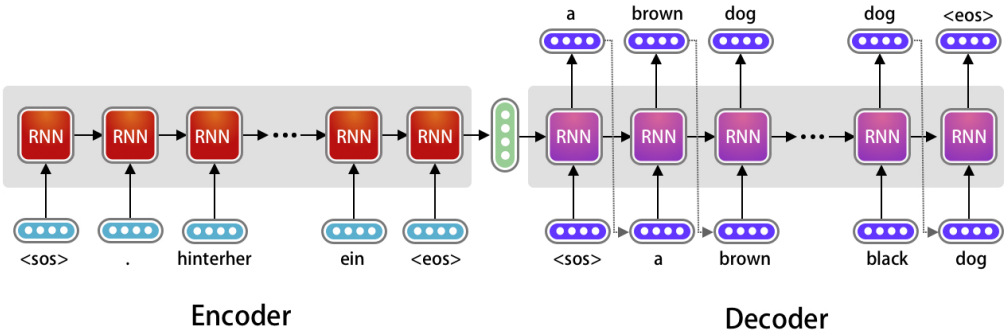
Challenges of RNNs

- **Vanishing/exploding gradients**: backpropagating through many time steps multiplies many derivatives
- Hard to learn long-range dependencies
- Fixes: **LSTM** (long short-term memory) and **GRU** (gated recurrent unit) cells

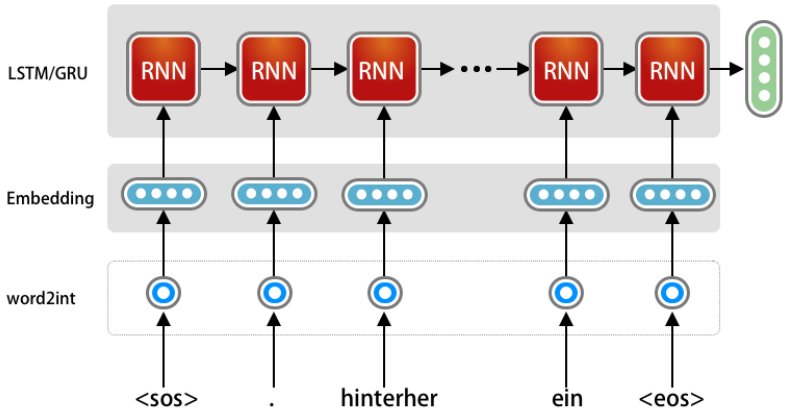
RNN for language translation



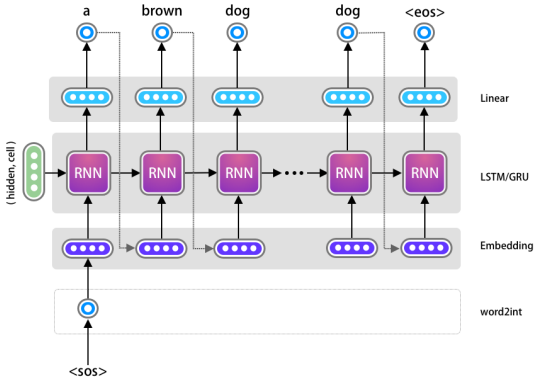
RNN for language translation



RNN for language translation



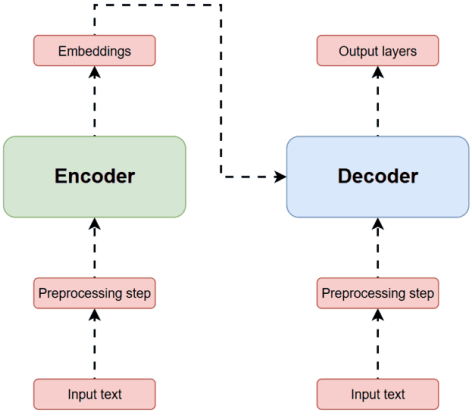
RNN for language translation



From RNNs to transformers

- RNNs process a sequence **one step at a time**: slow, and memory fades
- **Transformers** process the **entire sequence at once**, using **attention**
- Introduced in *"Attention Is All You Need"* (Vaswani et al., 2017)

Transformers: encoder–decoder



Key idea: attention

- Each token **looks at every other token** and decides which ones matter
- Example: in translation, each output word attends to the relevant source words
- For queries Q, keys K, values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

WEJ LAB
RUTGERS HEALTH
Center for Data Science

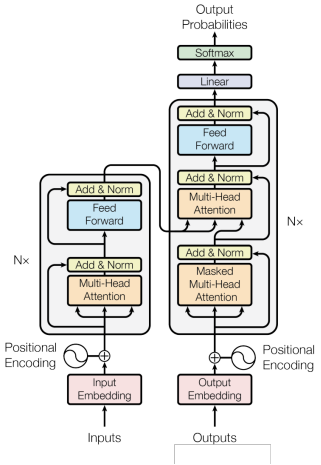
Transformer architecture

- **Multi-head** self-attention: several attention "heads" learn different relationships
- **Positional encoding** adds word order (there is no recurrence)
- Each block: attention + feedforward network, with residual connections and layer normalization
- Original transformer: **encoder + decoder** (translation). **BERT**: encoder only. **GPT**: decoder only.

Transformer architecture

BERT

Encoder



GPT

Decoder

WEJ LAB
RUTGERS HEALTH
Center for Data Science

GPT: a very advanced autocomplete

GPT = **G**enerative **P**retrained **T**ransformer. At its core, it predicts the next word (token):

- Looks at **all** the previous words at once
- Assigns **probabilities** to possible next words
- Picks a likely next word, then repeats

More on generative AI and LLMs in the coming weeks.

Summary

- A neuron: weighted sum + bias + non-linear activation; an MLP stacks layers of them
- Training: a loss, backpropagation (chain rule), and gradient descent
- Networks overfit easily: weight decay, early stopping, dropout; tune by CV
- On small tabular data (TB NanoString), an MLP matched but didn't beat SVMs
- CNNs share filters across images; RNNs carry memory along sequences
- Transformers replace recurrence with attention: the basis of today's LLMs

Python companion

Python versions of these examples (`scikit-learn`'s `MLPClassifier`):

`python/week7/neuralNet.ipynb`

WEJ LAB

RUTGERS HEALTH
Center for Data Science

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] pROC_1.18.5      nnet_7.3-19      neuralnet_1.44.2 lubridate_1.9.3
##  [5] forcats_1.0.0    stringr_1.5.1    dplyr_1.1.4      purrr_1.0.2
##  [9] readr_2.1.5      tidyr_1.3.1      tibble_3.2.1     tidyverse_2.0.0
## [13] caret_6.0-94     lattice_0.22-5   ggplot2_3.4.4
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.4         xfun_0.41            recipes_1.0.9
##  [4] tzdb_0.4.0           vctrs_0.6.5          tools_4.3.3
```